AI Inference, Simply Explained - Part 0 Notes

Video: AI inference as simply as possible. These notes set up the ideas used by the later code videos. The working version of my thoughts lives in `notes.md`; this is the version I would want beside me while following the series.

What am I doing?

I'm building a small LLM inference engine with C++ and CUDA to explain what happens between typing a prompt and receiving the next token. This project targets one particular model, Llama 3.2 1B Instruct; it is not a general vLLM replacement.

Here is the whole trip before we zoom in:

```
text prompt
  -> tokenizer -> token IDs
  -> embedding lookup -> hidden vectors
  -> 16 Transformer layers -> final hidden vector
  -> output scores for vocabulary tokens
  -> choose next token ID -> tokenizer decode -> text
```

These notes introduce the whole model before the implementation videos build it piece by piece. The later Part 6 snapshot (`ec066a7`) stopped after layer 0's attention residual; Part 7 completed all 16 layers and the first-token prediction. The current `src/main.cpp` also generates responses with a KV cache, batching, and paged decode attention. Read each companion note in the context of its video's milestone.

Wtf is inference?

The simplest answer I found is: **inference is using a trained model on new input to produce a prediction**. For a causal LLM, that prediction is usually a score for every possible next token. We pick a token from those scores, append it to the sequence, and run the model again to generate more text.

During **training**, an algorithm repeatedly adjusts the model's weights based on examples and an objective. During **inference**, we keep those learned weights fixed and use them in the model's operations. Loading a model file is therefore only the start of inference; the file does not execute itself.

I initially thought of a model as a file full of floating point numbers, which is a useful physical picture. The more complete picture is:

- **Architecture**: the blueprint saying which operations run, in which order, and with which shapes.
- **Weights**: the learned numbers those operations use. Our `model.safetensors` stores them.
- **Activations**: the temporary values produced for *this prompt* as it flows through the model. Unlike weights, these change from request to request.
- **Inference engine**: the program that loads the weights, implements the architecture, manages memory, and executes the operations.

One clarification from the working notes: the model predicts a **token**, not necessarily a whole word. A token can be a word, part of one, punctuation, or another text/byte piece. Also, an engine can affect the numerical result if it changes precision, operation order, or sampling. The main job is to execute the intended computation correctly and efficiently.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The chef analogy that helped me

Think of a chef learning recipes, and the kitchen being the engine:

- **Training** is culinary school. The chef practices and changes what they know. That learned knowledge is like the weights.
- **Inference** is asking the trained chef to make a meal from new ingredients. The chef uses what they learned to produce a result.
- **The inference engine** is the kitchen: equipment, space, and a process for getting ingredients to the chef and food back out. A better kitchen can serve more meals faster and use resources better.

There is a limit to the analogy. The real program follows exact numerical operations; it does not improvise the model architecture. We still need both the recipe/blueprint **and** the trained weights. A raw SafeTensors file cannot run like an executable, and an empty architecture has nothing learned to apply.

Why an inference engine at all?

Suppose I have the weight file. I still need code to:

1. Read its tensor metadata and put the weights where computation can use them.
2. Turn input text into the token IDs expected by this model.
3. Run operations in the model's exact order, including the right tensor shapes.
4. Keep intermediate values and, eventually, the attention KV cache in memory.
5. Choose a next token and translate its ID back to readable text.

Serving also brings the engineering problem: latency for one prompt, throughput across prompts, and GPU memory use. That is why the "engine" part matters even when the mathematical model is already trained.

Why C++ and CUDA?

Well, we wanna be fast and see how the hardware gets used. C++ handles the host-side control flow, file loading, tokenizer, allocations, and kernel launches. CUDA lets us express numerical work for an NVIDIA GPU. A lot of LLM work consists of matrix multiplications and other operations over huge arrays, which give a GPU plenty of parallel work.

This is not because every GPU task is faster than every CPU task. Moving data between CPU memory and GPU memory costs time. That is why we load large, mostly fixed model weights to VRAM once and keep intermediate activations there as operations run. We send small input token-ID arrays to the GPU when needed.

```
CPU / host RAM: open files, parse metadata, tokenize, schedule work
          | copy input and weights
          v
GPU / device VRAM: tensors and CUDA kernels doing model math
```

The CUDA getting-started notes unpack blocks, threads, shared memory, and launches before the kernel videos.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What the model is doing, in rough order

For the Llama 3.2 1B Instruct shape used in this project:

```
token IDs [T]
  -> embeddings [T, 2048]
  -> 16 decoder layers, each using attention and an MLP
  -> final RMSNorm [T, 2048]
  -> vocabulary scores [T, 128256]
  -> select a next-token ID
```

`T` is the number of prompt tokens. `128256` is the vocabulary size, and `2048` is the hidden-vector width. Each decoder layer updates those hidden vectors. The operations are *not* interchangeable: RMSNorm, Q/K/V projections, RoPE, masked attention, residual additions, and the MLP must run in the model's intended order. Part 1 has the architecture and weight names; Part 2 shows the first actual GPU operation.

There is a subtle point about the last step. The model produces **logits**, which are scores, not already-selected words. Greedy decoding takes the largest logit with `argmax`; other sampling strategies may choose differently. The shapes above show what the model could calculate at every prompt position. Our generation code applies the final norm and LM head only to the last prompt row, because that row predicts the first response token.

Prompt processing and generating the next token

Two phases will matter later:

- **Prefill**: run the whole prompt through the model, with causal attention, to establish hidden states and cache the K/V values for its positions.
- **Decode**: append one new token and run enough work to predict the next one, reusing the prior K/V cache rather than recomputing the whole prompt each time.

Why a **causal** mask? When predicting the next token, a position should only use tokens at that position or earlier. It must not peek into the future text. Part 8 explains how the completed prefill path retains each layer's K/V and how decode reuses them.

The main distinction to remember

```
training:  change the weights
inference: keep the trained weights fixed and calculate predictions
engine:    implement and run that calculation on hardware
```

That's the foundation for the rest of the series. We start with a file of learned numbers, then gradually build the executable path that gives those numbers a useful job.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.